



UNIVERSITY OF COPENHAGEN



Characterizing Learning in Deep Neural Networks using the lens of Complexity

Advanced Topics in Deep Learning

Raghavendra Selvan

Tenure-Track Assistant Professor
Dept. of Computer Science (ML Section)

raghav@di.ku.dk

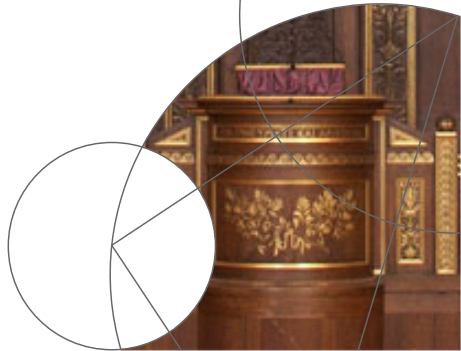
 [/raghavian](https://github.com/raghavian)

<https://raghavian.github.io>

Sustainable Artificial Intelligence for Sciences (SAINTS) Lab
<https://saintslab.github.io>

Collaborators: Pedram Bakhtiarifard, Tong Chen, Erik Dam,
Sophia Wilson & Jonathan Wenshøj.

September 3, 2026



Overview

- Recap
- From Occam's Razor to Algorithmic Complexity
- Tractable Estimation of Algorithmic Complexity
- Algorithmic Simplification of Neural Networks



Overview

- Recap
- From Occam's Razor to Algorithmic Complexity
- Tractable Estimation of Algorithmic Complexity
- Algorithmic Simplification of Neural Networks



Learning as a Tug of War

Every learning method **minimises** a sum of *two opposing terms*:

$$\text{objective}(h) = \underbrace{\text{loss of } h \text{ on } S \text{ (e.g. } \widehat{R}_S(h))}_{\text{fit term} \quad \downarrow \text{ as capacity } \uparrow} + \underbrace{\text{cost to specify } h \text{ (e.g. } \|\mathbf{w}\|, \text{ code length)}}_{\text{complexity term} \quad \uparrow \text{ as capacity } \uparrow}$$

where training data $S = \{(x_i, y_i)\}_{i=1}^n$; a candidate model $h \in \mathcal{H}$; training error $\widehat{R}_S(h) = \frac{1}{n} \sum_i \ell(h(x_i), y_i)$.

Topic-1 of ATDL

Can deep learning still be explained in this view? If not, are there other plausible explanations?



Most learning methods can be seen as pulling between **fit** and **complexity** terms

Framework	Fitness term	Complexity term
Regularised ERM	$\widehat{R}_S(h)$	$\lambda \Omega(h) \quad (\ \mathbf{w}\ _2^2, \ \mathbf{w}\ _1, \dots)$
Generalisation bounds	$\widehat{R}_S(h)$	$\sqrt{\text{capacity}(\mathcal{H})/n}$
PAC-Bayes	$\widehat{R}_S(Q)$	$\text{KL}(Q\ P)/n$
Bayesian evidence	$-\ln p(S \mid \hat{\mathbf{w}})$	$-\ln \frac{p(\hat{\mathbf{w}})}{\sqrt{\det H}}$ (Occam factor)
MDL / two-part code	$L(S \mid h)$ (residual bits)	$L(h)$ (model bits)

ERM: empirical risk minimisation · MDL: minimum description length · PAC: probably approximately correct.

Same framing everywhere: **fit** + **complexity**. A decreasing + an increasing term $\Rightarrow$ the best model sits at *finite* capacity, not at either extreme.

Rissanen, (1978), "Modeling by shortest data description"

Hinton et al., (1993), "Keeping the neural networks simple by minimizing the description length of the weights"

McAllester, (1999), "Some PAC-Bayesian Theorems"

For the Bayesian evidence, $-\ln p(S \mid \hat{\mathbf{w}})$ is the training negative log-likelihood (NLL; cross-entropy or mean-squared error) at the fitted weights $\hat{\mathbf{w}}$; $-\ln \frac{p(\hat{\mathbf{w}})}{\sqrt{\det H}}$ is

— $\ln$ of the Occam factor which prefers solutions in flat regions ($\det H$ small).



Six candidates for the **complexity** term

#	Paper	proposed complexity term for the trained network
1	Shwartz-Ziv & Tishby '17	$I(X; T)$: information the representation keeps about the input
2	Dziugaite & Roy '17	$\text{KL}(Q\ P)$: bits of the solution beyond the prior
3	Smith & Le '18	$\frac{1}{2} \ln \det H$: sharpness / (inverse) Bayesian evidence
4	Nakkiran et al. '21	effective model complexity vs. n (measured, not derived)
5	Liu et al. '22	(dis)order of the learned representation (effective theory)
6	Sakabe et al. '26	$\mathcal{C}(\mathbf{w})$: algorithmic complexity of the weights

All are **description-length-flavoured** (Blier et al. 2018) . None is proven to be *the* one; and none covers every phenomenon.



Overview

- Recap
- From Occam's Razor to Algorithmic Complexity
- Tractable Estimation of Algorithmic Complexity
- Algorithmic Simplification of Neural Networks



The curious case of Deep Learning

- **Double descent:** test error falls, then *rises* at the interpolation threshold $p \approx n$, then falls again; the bias–variance U is only the left half (Belkin et al. 2019) (Nakkiran et al. 2021) .
- **Grokking:** the **fit** term hits 0 in a few hundred steps; generalisation appears 10^2 – 10^4 steps later, while a second quantity (weight norm, representation structure) keeps shrinking (Power et al. 2022) (Liu et al. 2022) .
- **Benign overfitting:** $\hat{R}_S(\hat{h}) = 0$ on noisy data, yet $R(\hat{h}) \rightarrow R^*$ (Bartlett et al. 2020) .
- **Implicit bias:** gradient descent on separable data $\rightarrow$ the *max-margin* solution; the optimiser, not the loss, picks h (Soudry et al. 2018) .

In every case: *effective complexity* of the trained $\hat{h} \neq$ *nominal complexity* (p, d_{VC}) of $\mathcal{H}$.

Belkin et al., 2019, “Reconciling modern machine-learning practice and the classical bias–variance trade-off”

Nakkiran et al., 2021, “Deep double descent: Where bigger models and more data hurt”

Power et al., 2022, “Grokking: Generalization beyond overfitting on small algorithmic datasets”

Liu et al., 2022, “Towards Understanding Grokking: An Effective Theory of Representation Learning”

Bartlett et al., 2020, “Benign overfitting in linear regression”

Soudry et al., 2018, “The implicit bias of gradient descent on separable data”



Challenge in DL: Choosing the best among the interpolating models

Among the infinitely many $\{h : \widehat{R}_S(h) = 0\}$

How do we choose the models that generalise?

Before the formal answers: Let's do a short detour.



“Prefer the simpler explanation” is old and cross-cultural

Most famously captured as:



William of Ockham

- **Greece.** Aristotle: “the more limited, if adequate, is always preferable”; “nature does nothing in vain”.
- **India.** The Nyāya and Mīmāṃsā schools argue by *lāghava* (parsimony) against *gaurava* (complexity): the account that postulates fewer entities or steps wins.
- **Medieval Europe.** Maimonides, Duns Scotus, and William of Ockham: “*plurality is never to be posited without necessity*”.
- **Early modern science.** Newton, *Principia*, Rule I: “admit no more causes. . . than are both true and sufficient”.

Occam's Razor (14th Cen.)

Entities must not be multiplied beyond necessity

Information Theorists formalized this in the 1960's

Let us next see how.



Algorithmic Complexity of Binary Objects

Consider the two sequences:

- $A = [100111010110]$
- $B = [100100100100]$

Which sequence is more complex?

Seq. A

```
print "100111010110"
```

Seq. B

```
[print "100"] x 4
```

Implication: Random strings are not compressible. Structured strings can be compressed.

Kolmogorov-Chaitin-Solomonoff (KCS) Complexity

The algorithmic complexity of a finite object $\mathbf{w} \in \{0, 1\}^*$, is the length of the shortest program p among *all possible* programs that outputs $\mathbf{w}$ and halts.

$$\mathcal{C}(\mathbf{w}) = \min\{|p|, U(p) = \mathbf{w}\} \quad (1)$$

where U is a fixed universal prefix-free Turing machine and $|p|$ denotes the length of the program in bits.

Hypothesis: Can this notion of complexity relate to learning?

Solomonoff, 1960, *A preliminary report on a general theory of inductive inference*

Kolmogorov, 1965, "Three approaches to the quantitative definition of information"

Chaitin, 1966, "On the length of programs for computing finite binary sequences"



From $\mathcal{C}$ to MDL: Restrict the description method

- $\mathcal{C}(\mathbf{w}) = \min\{|p| : U(p) = \mathbf{w}\}$ minimises over *every* program: it is uncomputable, and approachable only from *above*.
- **MDL** keeps the “shortest description” idea but replaces “all programs” with a *computable family of models* $\mathcal{M}$ (polynomials, Markov chains, a net architecture, `gzip`, ...). Describe $\mathbf{w}$ in *two parts*: which model $h \in \mathcal{M}$, then $\mathbf{w}$ under h (codelength $-\log_2 P(\mathbf{w} | h)$):

$$L_{\mathcal{M}}(\mathbf{w}) = \min_{h \in \mathcal{M}} \left[\underbrace{L(h)}_{\text{complexity: model bits}} + \underbrace{(-\log_2 P(\mathbf{w} | h))}_{\text{fit: data-given-model bits}} \right].$$

- “load h , then decode the residual” is a program $\Rightarrow \mathcal{C}(\mathbf{w}) \leq L_{\mathcal{M}}(\mathbf{w}) + c_{\mathcal{M}}$: MDL is an **upper bound** on $\mathcal{C}$.

What happens if we increase the richness of $\mathcal{M}$?

Richer $\mathcal{M}$: tighter bound. $\mathcal{M} = \{\text{all programs}\}$, $L(h) = |h|$ gives back $\mathcal{C}$ (and its uncomputability).

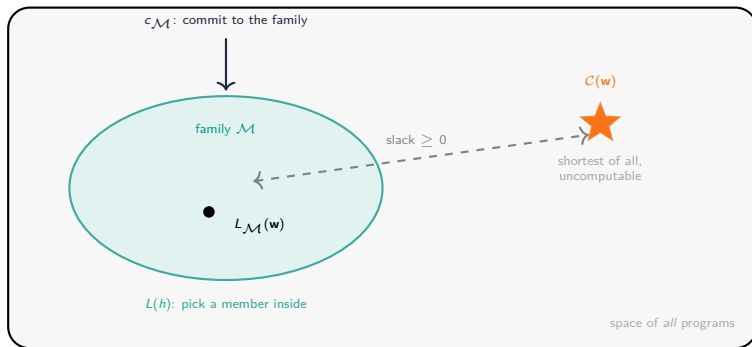
$c_{\mathcal{M}}$: the length of a fixed decoder for $\mathcal{M}$ on the reference machine (parse h , rebuild $P(\cdot | h)$, run an arithmetic decoder). Constant in $\mathbf{w}$, so it washes out for large objects; depends on $\mathcal{M}$ and the machine.

Rissanen, (1978), “*Modeling by shortest data description*”

Grünwald, (2007), *The Minimum Description Length Principle*



MDL narrows down the space of all programs



- Two bit-costs: $c_{\mathcal{M}}$ to say *which* family (fixed, paid once), then $L(h)$ to pick a member inside it, plus the residual.
- The shortest program may live anywhere in the space of all programs
- MDL only ever reaches the best point of the sub-space it chose, so $L_{\mathcal{M}}(\mathbf{w}) + c_{\mathcal{M}} \geq C(\mathbf{w})$.



MDL: Learning is Compression

Take the model family to be the hypothesis class $\mathcal{H}$: the “model” is a hypothesis $h \in \mathcal{H}$ (its parameters), the “residual” is the labels given h :

$$L(S) = \underbrace{L(h)}_{\text{complexity (model bits)}} + \underbrace{L(S | h)}_{\text{fit (residual bits)}} \quad (\text{two-part code})$$

- The Tug of War, now measurable in: **bits**.
- A model that memorises pays for it: $L(h)$ carries every bit it stored.
- A model that found real structure has a short $L(h)$ *and* a short $L(S | h)$.
- Compression is *sufficient* for generalisation: sending n labels in $\ll n$ bits means you found regularity that transfers.

For a neural network, h is its weights $\mathbf{w}$. $L(S | h)$ is just the training loss; the hard term is $L(\mathbf{w})$: how do you code real-valued weights, with $p \gg n$?



Coding Real-valued Weights

$L(\mathbf{w})$ is the length of a **code** for the weights. Fix that code before the data and call it the **prior** P : a specific $\mathbf{w}$ then costs $L(\mathbf{w}) = -\log_2 P(\mathbf{w})$ bits.

- **But $\mathbf{w}$ is real-valued.** Naming one point $\hat{\mathbf{w}}$ *exactly* needs infinite precision, hence infinitely many bits.
- **We only need the *region*** of weights that fit the data equally well (a flat basin).
- **Idea:** Code a distribution $Q(\mathbf{w})$ over that region; its cost relative to the prior is finite:

$$L(\mathbf{w}) = \text{KL}(Q \| P) \quad \left(\rightarrow -\log_2 P(\hat{\mathbf{w}}) \text{ as } Q \text{ concentrates} \right).$$

A wider basin $\Rightarrow$ wider $Q \Rightarrow$ smaller KL.

$$L(S) = \underbrace{n \hat{R}_S(Q)}_{\text{fit} = L(S | h)} + \underbrace{\text{KL}(Q \| P)}_{\text{complexity} = L(\mathbf{w})} \quad \min_Q : \text{variational inference}$$

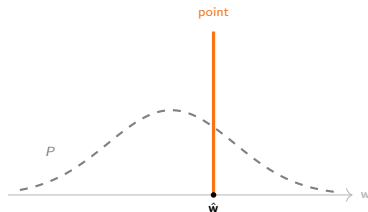
Hinton et al., (1993), “Keeping the neural networks simple by minimizing the description length of the weights”

Hochreiter et al., (1997), “Flat minima”

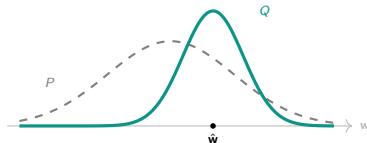
Graves, (2011), “Practical variational inference for neural networks”



$L(\mathbf{w})$: Specifying a unique model vs. Committing to a Distribution



pin $\hat{w}$ exactly
 $-\log_2 P(\hat{w})$ bits,
 precision $\rightarrow \infty$



describe a spread Q
 $KL(Q\|P)$ bits:
 how far Q sits from P

$KL(Q\|P) \rightarrow -\log_2 P(\hat{w})$ as Q narrows to the point; $KL(Q\|P) \rightarrow 0$ as $Q \rightarrow P$. Training sits in between.



Story so far...

- We are seeking useful measures of model **complexity** , that can help identify generalisable models
- KCS complexity measures the intrinsic complexity of objects, but is uncomputable
- MDL approximates KCS complexity from above; by narrowing down the program space
- Model complexity term ends up taking the shape of $\text{KL}(Q\|P)$



Papers 2 and 3: the same **complexity** term, two uses

We have $L(S) = n\hat{R}_S(Q) + \text{KL}(Q\|P)$. Papers 2 and 3 each take the $\text{KL}(Q\|P)$ term and work with it:

- **Paper 2 (Dziugaite & Roy):** $\frac{1}{n}L(S) = \hat{R}_S(Q) + \frac{1}{n}\text{KL}(Q\|P)$ *upper-bounds the test error* (PAC–Bayes). Minimise it by SGD and a neural network with $p \gg n$ gets a *non-vacuous* number
- **Paper 3 (Smith & Le):** $\min_Q L(S) = -\log_2 p(S)$, the *Bayesian evidence*. Its Laplace approximation is the Occam factor $\frac{1}{2} \ln \det H$, so a *flat* minimum = a short code; mini-batch noise is what lands SGD there.

The **complexity** term is a description length, measured against a *hand-picked* prior P .

Its assumption-free limit is $\mathcal{C}(\mathbf{w})$: uncomputable. Can we estimate it?



Overview

- Recap
- From Occam's Razor to Algorithmic Complexity
- Tractable Estimation of Algorithmic Complexity
- Algorithmic Simplification of Neural Networks



Approximating Algorithmic Complexity with Algorithmic Information Theory

- KCS Complexity is uncomputable¹
- Approximate $\mathcal{C}$ using algorithmic probability and the Coding Theorem
- Algorithmic probability is the probability that a universal Turing machine U will produce $\mathbf{w}$ and halt.

$$P(\mathbf{w}) = \sum_{P_{U(p)=\mathbf{w}}} 2^{|-p|}$$
- Coding Theorem (Levin, 1974) relates algorithmic complexity and algorithmic probability:

$$\mathcal{C}(\mathbf{w}) = -\log_2 P(\mathbf{w}) + O(1)$$
- By exhaustively generating all n -state, k -symbol Turing machines, we obtain the empirical output distribution for producing $\mathbf{w}$. Let D_n denote the normalized frequency with which these machines halt and produce $\mathbf{w}$. Then

Coding Theorem Method (CTM)

Approximate $\mathcal{C}(\mathbf{w})$ by

$$\mathcal{C}_{\text{CTM}}(\mathbf{w}) = -\log_2 D_n(\mathbf{w}) \approx \mathcal{C}(\mathbf{w}), \quad (2)$$

i.e. CTM based estimation approximates $\mathcal{C}(\mathbf{w})$ up to an additive constant.

¹More precisely, $\mathcal{C}$ is lower semi-computable i.e. cannot be computed with absolute certainty but can be arbitrarily approximated from above.

Levin, 1974, "Laws of information conservation (nongrowth) and aspects of the foundation of probability theory"

Zenil, Soler-Toscano, et al., 2015, *Two-Dimensional Kolmogorov Complexity and Validation of the Coding Theorem Method by Compressibility*



CTM does not scale to large objects.

- Exhaustive enumeration of n -state Turing machines is feasible only for small n (typically $n \leq 5$)²
- CTM tables are currently available only for short 1D binary strings (up to 13 bits) and small 2D binary blocks (4×4)
- **Block Decomposition Method**(BDM) extends CTM to larger objects by partitioning them into smaller blocks, estimating the complexity of each block using pre-computed CTM tables, and accounting for repeated blocks with a logarithmic multiplicity term.
Concretely, let $\mathbf{w}$ be a large object partitioned by a lossless function $\Pi(\mathbf{w}) = (w_1, \dots, w_m)$ and let $\Pi_u(\mathbf{w})$ denote the set of distinct blocks of $\mathbf{w}$. The KCS complexity is then estimated as

$$\mathcal{C}_{\text{BDM}}(\mathbf{w}) = \sum_{w \in \Pi_u(\mathbf{w})} (\mathcal{C}_{\text{CTM}}(w) + \log_2 n_w). \quad (3)$$

where n_w is the number of occurrences of $w \in \Pi(\mathbf{w})$ and $\mathcal{C}_{\text{CTM}}(w)$ is the estimated KCS complexity of w , derived from the CTM table.

²Since the number of transition tables N grows exponentially with the number of states as $N(n) = (4n + 2)^{2n}$.

Zenil, Hernández-Orozco, et al., 2018, "A decomposition method for global evaluation of Shannon entropy and local estimations of algorithmic complexity"



Estimating Algorithmic Complexity: The Recipe

- True KCS complexity is uncomputable, by definition
- Coding theorem relates algorithmic probability with algorithmic complexity
- CTM simulates Turing machines to obtain empirical algorithmic probability. Stored as CTM tables
- BDM scales it to larger objects by using CTM estimates of smaller blocks and counting the number of occurrences

Caveats

- BDM primarily works for binary objects
- Does not capture long-range patterns
- Linking it to learning is the next step (Paper 6)



Overview

- Recap
- From Occam's Razor to Algorithmic Complexity
- Tractable Estimation of Algorithmic Complexity
- Algorithmic Simplification of Neural Networks



Paper 6: Sakabe et al., BDM on a trained network's weights

The first of the six papers to *estimate* the **complexity** = $\mathcal{C}(\mathbf{w})$ term directly.

- **Object.** Binarise the trained weights, then apply BDM: $\mathcal{C}_{\text{BDM}}(\mathbf{w})$. This measures the *model-bits* term of MDL alone, with no two-part code and no residual.
- **Controlling fit.** Look only at the pre-overfitting regime (validation loss still flat), so the network is not memorising.
- **Finding.** $\mathcal{C}_{\text{BDM}}(\mathbf{w})$ *decreases* through training there: SGD drifts toward algorithmically *simple* weights, and $\mathcal{C}_{\text{BDM}}$ tracks the loss better than the Shannon entropy of the same bits.

A *descriptive* result: the $\mathcal{C}(\mathbf{w})$ term really does fall as a network learns (the **fit** + **complexity** picture of Papers 1 and 5, now for algorithmic complexity).

Caveats: binarisation is lossy; BDM misses long-range structure; *why* SGD does this is open.

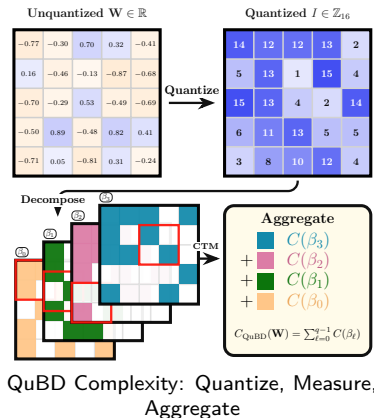


Algorithmic Simplification of Neural Networks



A Tractable Estimator of Algorithmic Complexity

- Paper 6 binarised the weights first: BDM works, but a 1-bit binarisation discards most of the object
- Quantized Block Decomposition method (QuBD) Complexity
- Quantize original objects to a finite alphabet
- Represent each quantized object by its binary expansion across bit-planes
- QuBD complexity obtained by aggregating CTM-based estimations over all the bit-planes
- Yields a tighter estimate of KCS complexity than BDM



QuBD: A Tractable Estimator of Algorithmic Complexity

Let $\mathbf{w} = (w_1, \dots, w_d) \in \mathbb{R}^d$ denote a d -dimensional real object. We map $\mathbf{w}$ to a finite alphabet Σ_b using an affine uniform quantizer $Q_b : \mathbb{R}^d \rightarrow \Sigma_b^d$ yielding the discrete object

$$\mathbf{q} = Q_b(\mathbf{w}) = (q_1, \dots, q_d) \in \Sigma_b^d, \text{ with } \Sigma_b := \{0, \dots, 2^b - 1\}, \quad (4)$$

where each q_i is a b -bit symbol of 2^b possible elements. The bit-plane representation uses the binary expansion of each quantized weight

$$q_i = \sum_{\ell=0}^{b-1} 2^\ell \beta_{i,\ell}, \quad \beta_{i,\ell} \in \{0, 1\}, \quad \beta_\ell := (\beta_{1,\ell}, \dots, \beta_{d,\ell}) \in \{0, 1\}^d \quad (5)$$

where $\beta_{i,b-1}$ denotes the Most Significant Bit (MSB) and $\beta_{i,0}$ denotes the Least Significant Bit (LSB) of q_i . The vector β_ℓ collects the ℓ -th bit of each of the d symbols, forming the ℓ -th bit-plane.

QuBD Complexity from Bit-Planes

$$C_{\text{QuBD}} = \sum_{\ell=0}^{b-1} C_{\text{BDM}}(\beta_\ell) \quad (6)$$



KCS complexity between the target object and the estimated quantized object

Using b^* for the target lossless precision, and $\mathbf{w}^*$ for the corresponding target object. The representation is lossless when $b = b^*$ since the bit-planes jointly and uniquely determine $\mathbf{w}^*$.

For, $b < b^*$, denote the omitted lower-bit residuals:

$$\delta_b := \mathbf{w}^* - 2^{(b^*-b)}\mathbf{q}. \quad (7)$$

Since $\mathbf{q}$ and δ_b capture the retained and omitted information in the b bit planes, respectively, the target object can be estimated as the composite $(\mathbf{q}, \delta_b)$.

Proposition (Residual Decomposition)

For a d -dimensional target object $\mathbf{w}^$ with precision $1 \leq b \leq b^*$,*

$$\mathcal{C}(\mathbf{w}^*) = \mathcal{C}(\mathbf{q}) + R_b + O(\log b^* d), \quad R_b := \mathcal{C}(\delta_b \mid \mathbf{q}), \quad (8)$$

where $0 \leq R_b \leq (b^ - b)d + O(1)$. So, estimating $\mathbf{w}^*$ from $\mathbf{q}$ misses the complexity due to the residual information R_b , up to a logarithmic term. If $b = b^*$, then $\delta_b = 0$ and $R_b = O(1)$.*

When $b = 1$, BDM misses the complexity due to the R_b .



Normalized QuBD Complexity

We normalize QuBD complexity relative to a random object of the same length ($\tilde{\mathbf{w}}$), denoted $\Delta\mathcal{C}_{\text{QuBD}}$, for easier interpretation:

$$\Delta\mathcal{C}_{\text{QuBD}} = \frac{\mathcal{C}_{\text{QuBD}}(\mathbf{w})}{\mathcal{C}_{\text{QuBD}}(\tilde{\mathbf{w}})}. \quad (9)$$

A truly random object has $\Delta\mathcal{C}_{\text{QuBD}} \approx 1$, whereas an algorithmically simple object³ has $\Delta\mathcal{C}_{\text{QuBD}} \approx 0$.

³Strictly speaking, $\Delta\mathcal{C}_{\text{QuBD}} \approx 0$ holds only for sufficiently large objects, as even maximally simple objects such as an all-zero string retain some complexity. For objects of sizes (8×8) , (16×16) , (32×32) , $\Delta\mathcal{C}_{\text{QuBD}}$ for an all zeros string is 0.19, 0.05, and 0.01, respectively.

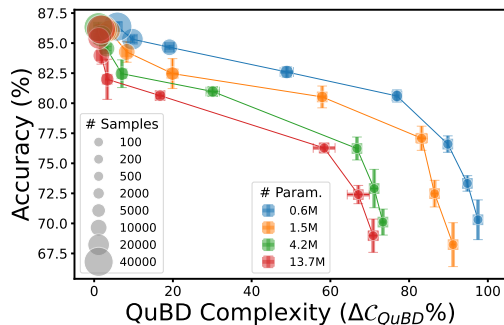


Algorithmic Complexity and Learning/Compression in Deep Learning

Hypothesis: Trained models can be compressed because they have low algorithmic complexity.



What happens to the algorithmic complexity of the weights as networks learn?

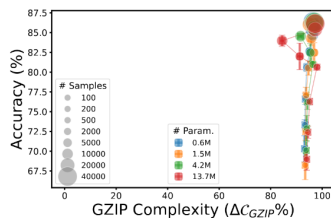
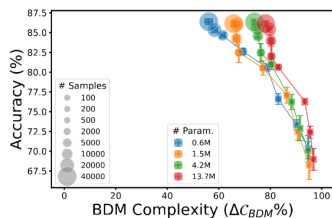
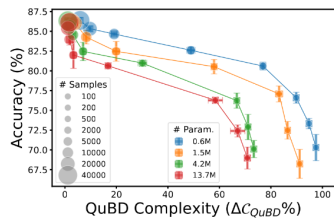


Multi-layered perceptrons of varying complexity trained and validated (10k) on FashionMNIST.

Learning Reduces Algorithmic Complexity of Deep Neural Networks



QuBD Complexity is more discernible than BDM/GZIP



Algorithmic Complexity Tracks Generalization and Grokking

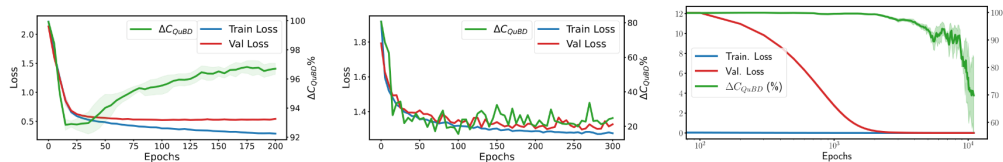
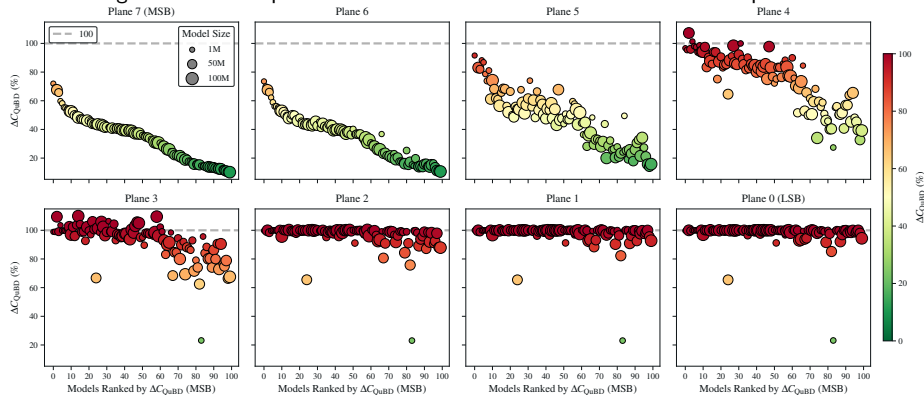


Figure 5: Learning curves along with ΔC_{QuBD} evolution for MLP (**left**) and Tiny ViT (**center**). Algorithmic Complexity during (**right**) Grokking for the Modulo operator for $P = 97$.

QuBD Complexity is expressive across model sizes

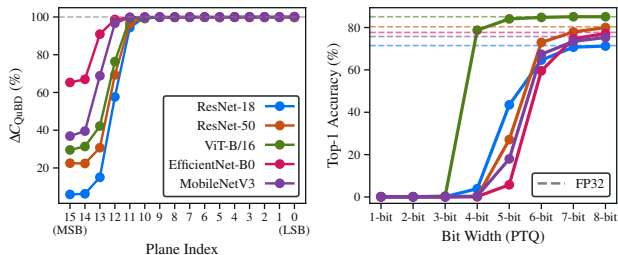
Using $b = 8$ for 100 pretrained models with size 1M to 100M trainable parameters.



QuBD captures how useful bit planes are.



QuBD Complexity as a diagnostic for compression



High complexity bit planes can be discarded (post-training quantization)

QuBD as a diagnostic tool

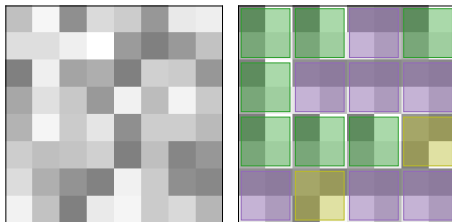
Empirical evidence for generalisation, grokking, training dynamics.

What else can we do?



Algorithmic Simplification of Neural Networks

- BDM measures complexity of small, repeating blocks.
- Low algorithmic complexity implies few blocks that are repeated
- Conversely, can we induce this low complexity during training?
- *Mosaic-of-Motifs* (MoMos): a constrained parameterization of model weights
- Partitioning parameters into blocks of size s , it restricts each block to be selected from a set of k reusable motifs, specified by a reuse pattern $\mathcal{M}$, referred to as a mosaic.



An unconstrained (left) 8×8 parameterization with randomly selected motifs highlighted. MoMos parameterization (right) enforcing 2×2 motif reuse

MoMos Algorithm

Algorithm 1 MoMos Optimization

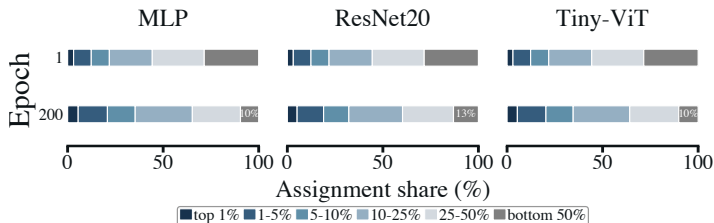
input initialization $\mathbf{w}_0 \in \mathbb{R}^n$, block size s , model capacity $c \in [0, 1]$, iterations n_T , data $\mathcal{D}$, loss ℓ , optimizer OPT

output $\hat{\mathbf{w}}^{(n_T)} \in \mathcal{H}_{s,k}$, motifs $\mathcal{Z}^{(n_T)}$, mosaic $\mathcal{M}^{(n_T)}$

- 1: $m \leftarrow \lceil n/s \rceil$, $k \leftarrow \max\{1, \lfloor c \cdot m \rfloor\}$
- 2: **for** $t = 1, \dots, n_T$ **do**
- 3: $\mathbf{w}^{(t)} \leftarrow \text{OPTSTEP}(\mathbf{w}_{t-1}; \mathcal{D}, \ell)$
- 4: $(\mathbf{b}_1^{(t)}, \dots, \mathbf{b}_m^{(t)}) \leftarrow \psi(\mathbf{w}^{(t)})$
- 5: $\mathbf{z}_1^{(t)} \leftarrow \mathbf{0}_s$
- 6: **if** $k > 1$ **then**
- 7: Uniformly sample distinct indices $j_1, \dots, j_{k-1}$ from $[m]$
- 8: **for** $r = 2, \dots, k$ **do**
- 9: $\mathbf{z}_r^{(t)} \leftarrow \mathbf{b}_{j_{r-1}}^{(t)}$
- 10: **end for**
- 11: **end if**
- 12: $\mathcal{Z}^{(t)} \leftarrow (\mathbf{z}_1^{(t)}, \dots, \mathbf{z}_k^{(t)})$
- 13: **for** $i = 1, \dots, m$ **do**
- 14: $\mathcal{M}^{(t)}(i) \leftarrow \arg \min_{r \in \{1, \dots, k\}} \|\mathbf{b}_i^{(t)} - \mathbf{z}_r^{(t)}\|_2^2$
- 15: **end for**
- 16: $\hat{\mathbf{w}}^{(t)} \leftarrow \psi^{-1}(\mathbf{z}_{\mathcal{M}^{(t)}(1)}^{(t)}, \dots, \mathbf{z}_{\mathcal{M}^{(t)}(m)}^{(t)})$
- 17: $\mathbf{w}_t \leftarrow \hat{\mathbf{w}}^{(t)}$
- 18: **end for**



MoMos Yields Algorithmically Simpler, Compressible Models



For comparable performance with dense, unconstrained models.



Summary & Conclusions

- Two forces at work in ML algorithms (**fit** and **complexity**)
- Deep learning models have shown to generalise even after achieving zero training error
- We are seeking a **complexity** functional that can identify these generalisable hypotheses
- Algorithmic complexity approximated using MDL and operationalised as KLD can be useful
- Recent attempts at estimating algorithmic complexity (BDM)
- QuBD offers better estimates than existing methods like BDM
- But, discards any inter-plane structure; still relies on CTM tables.
- *Diagnostic* approach to characterizing deep learning with QuBD
- *Prescriptive* ideas inspired from AIT; MoMos as a constrained parameterization
- Ongoing work on complexity-informed optimization, data curation.
- Open questions on formal connection between algorithmic simplification and generalisation



Links to QuBD and MoMos papers.

AIT could offer a unified lens to understand compression and learning in deep learning.



Funding received under European Union's Horizon Europe Research and Innovation programme under grant agreement No. 101019777



Funding received under Independent Research Fund Denmark (DF) under grant 90320043H



Backup: Turing machines and prefix codes

Turing machine. An abstract computer: an infinite tape, a read/write head, a finite control. A *program* p is an input string; $U(p)$ is the output if the machine halts. A **universal** U simulates any other machine from its description, so $\mathcal{C}(\mathbf{w}) = \min\{|p| : U(p) = \mathbf{w}\}$ is machine-independent up to an additive constant.

Prefix code (a.k.a. **prefix-free code**). No codeword is a prefix of another, so the decoder reads left to right and knows exactly where each codeword ends: it is *self-delimiting*, no separators needed.

$\{0, 10, 110, 111\}$ is prefix-free; $\{0, 01, 11\}$ is not (0 is a prefix of 01).

Kraft's inequality. Prefix-code lengths satisfy $\sum_i 2^{-\ell_i} \leq 1$, and any lengths satisfying it are realisable by some prefix code. Hence $2^{-\ell(\mathbf{w})}$ is a (sub-)probability: a code $\equiv$ a distribution, $\ell(\mathbf{w}) = -\log_2 P(\mathbf{w})$.

Prefix-free U . If U 's halting programs form a prefix-free set, program lengths obey Kraft too ($\sum_p 2^{-|p|} \leq 1$). That is what makes $2^{-|p|}$ a probability (algorithmic probability) and $\mathcal{C}$ a genuine codelength.



A refresher: information, measured in bits

Fix a distribution p . The *ideal code length* of an outcome x is $-\log_2 p(x)$ bits (Kraft / Shannon). Everything below is an *average* code length.

- **Entropy** $\mathbb{H}(X) = \mathbb{E}_p[-\log_2 p(X)]$
bits to encode one draw from p .
- **Conditional** $\mathbb{H}(X | Y) = \mathbb{E}[-\log_2 p(X | Y)]$
bits left in X once Y is known.
- **Mutual information**

$$I(X; Y) = \mathbb{H}(X) - \mathbb{H}(X | Y) = \text{KL}(p_{XY} \parallel p_X p_Y) \geq 0$$

bits Y carries about X . *Paper 1's complexity* is $I(X; T)$: bits the representation T keeps about the input.

- **Relative entropy (KL)**

$$\text{KL}(q \parallel p) = \mathbb{E}_q \left[\log_2 \frac{q(X)}{p(X)} \right] \geq 0$$

extra bits from coding q -data with a p -code.

- **Cross-entropy**

$$\mathbb{E}_q[-\log_2 p(X)] = \mathbb{H}(q) + \text{KL}(q \parallel p)$$

the training **fit** term (NLL) is the empirical cross-entropy.

Every quantity is an expected code length $-\log_2 p$. Shannon needs the distribution p and *averages*. Kolmogorov complexity $\mathcal{C}(\mathbf{w})$ drops p : the shortest *program* that outputs the single object $\mathbf{w}$. They agree on average ($\mathbb{E}_p[\mathcal{C}(X)] = \mathbb{H}(X) + \mathcal{O}(1)$ for a computable source), but $\mathcal{C}$ scores one object, e.g. one weight vector $\mathbf{w}$.

Shannon, (1948), "A mathematical theory of communication"

Cover et al., (2006), *Elements of Information Theory*



What is the prior P , and where does it come from?

- The **model class** $\mathcal{H}$ is the *architecture* (fixed layer sizes, connectivity); each weight vector $\mathbf{w}$ gives a predictive distribution $P(S \mid \mathbf{w})$.
- $P(\mathbf{w})$ is the **code over the weights** of that architecture: the two-part code's “which model” term $-\log_2 P(\mathbf{w})$, softened to $\text{KL}(Q \parallel P)$. Like $L(h)$ in generic MDL, it is a *design choice* fixed before the data.

Two standard choices:

- $P = \mathcal{N}(\mathbf{0}, \lambda I)$: small weights are cheap $\Rightarrow$ exactly L_2 weight decay.
- $P = \mathcal{N}(\mathbf{w}_0, \lambda I)$ centred at the random init $\mathbf{w}_0$: then $\text{KL}(Q \parallel P)$ counts the bits *training added* to the weights. Paper 2 uses this, with λ tuned without the training labels.

A “better” P (mass where good solutions sit) gives shorter codes, but you may not choose it using the training data.

